

Parallel Hybrid Retrieval: Enhancing Temporal RAG via Concurrent Graphiti and Semantic Search

1st Muhammad Ibrahim Kiani

Department of AI & DS

FAST NUCES

Islamabad, Pakistan

i232536@isb.nu.edu.pk

2nd Muhammad Noor

Department of AI & DS

FAST NUCES

Islamabad, Pakistan

i232520@isb.nu.edu.pk

Abstract—Standard Retrieval-Augmented Generation (RAG) architectures face significant challenges when processing corpora with complex temporal dynamics, often leading to temporal hallucinations where models retrieve outdated factual states. Furthermore, querying multi-modal indexes sequentially introduces critical inference latency. This paper introduces a dual-retrieval RAG architecture that harmonizes traditional semantic search over chunked documents with Temporal Knowledge Graphs (Graphiti) via parallel processing. By merging these paradigms concurrently, the proposed system ensures accurate semantic retrieval while maintaining historical state awareness, effectively mitigating state-based hallucinations while minimizing inference bottlenecks.

Index Terms—RAG, Temporal Knowledge Graphs, Graphiti, Semantic Search, Parallel Processing, Local State Hallucination

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated exceptional capabilities in generating human-like text and reasoning over vast quantities of data. The advent of Retrieval-Augmented Generation (RAG) has allowed these models to bypass strict context-window limitations by dynamically pulling relevant context from external databases. However, standard RAG systems treat information statically. They commonly rely on chunking text into independent vector embeddings, stripping the relational and chronological links that define how information evolves over time.

When applied to environments characterized by shifting states or facts (e.g., medical records, changing legal documents, or evolving storylines), traditional semantic search struggles. A model might access a document verifying a past fact but completely fail to retrieve the subsequent document that invalidates it. This issue is termed the “hallucination of state,” where an AI recalls a valid prior fact while failing to recognize its expiration.

While Knowledge Graphs (KGs)—and specifically Temporal Knowledge Graphs like Graphiti—have emerged to solve this lack of relational context, querying complex hierarchical graphs alongside dense vector stores sequentially generates severe inference bottlenecks.

This research proposes a **Parallel Hybrid Retrieval** framework that concurrently queries both a semantic vector database and a Graphiti-backed temporal knowledge graph. Our approach effectively solves the hallucination of state by dynam-

ically confirming the temporal validity of retrieved semantic chunks against the KG, parallelizing the retrieval streams to maintain strict latency limits.

II. LITERATURE REVIEW / RELATED WORK

A. Standard Hybrid RAG

Hybrid RAG systems traditionally combine dense retrieval (e.g., embeddings via BGE or OpenAI) with sparse retrieval (e.g., BM25) to capture both semantic meaning and exact keyword matches (arXiv:2408.04948). While effective for general QA, these methods do not encode temporal dependencies. A chunk containing the sentence “The CEO of the company is John” is treated with the same validity as a later chunk stating “Jane has been appointed as the new CEO,” leading the LLM to conflicting context or hallucinations.

B. Temporal Knowledge Graphs & Graphiti

Knowledge Graphs map entities and relationships as nodes and edges. Temporal KGs extend this paradigm by attaching temporal validity scopes (start and end times) to edges. Recent advancements, such as Graphiti (arXiv:2501.13956), allow for the construction of dynamic graphs where relationships decay, expire, or get overridden by new data. Graphiti ensures that models can perform temporal reasoning, understanding that a fact was true *then* but is false *now*. However, utilizing exclusively graph-based retrieval degrades verbatim text generation compared to raw semantic document retrieval.

C. Parallelization in RAG Architectures

Traditional multi-index RAG architectures employ a sequential router, deciding which index to query first, or querying them consecutively. This leads to latency stacking. Recent exploration into Parallel Data Computing (PDC) architectures points to utilizing multithreading or asynchronous I/O to simultaneously hit multiple diverse data stores, merging the unified context right before generation. Our work adapts these PDC methodologies specifically for merging temporal graphs with vector stores.

III. PROPOSED METHODOLOGY

The proposed architecture adopts a dual-stream concurrent processing model to handle both the textual nuance of the document and its temporal state. The methodology is split into four primary stages: Ingestion, Parallel Retrieval, Aggregation, and Generation/Evaluation.

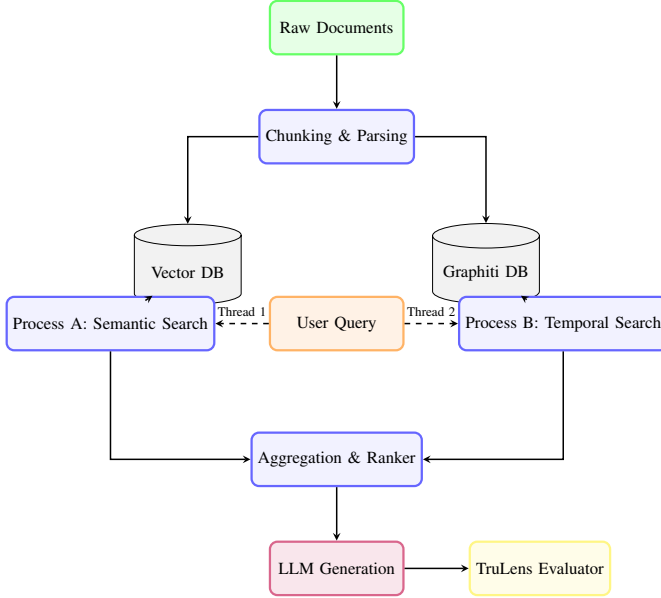


Fig. 1. System Architecture of the Parallel Hybrid Temporal RAG.

A. Ingestion & Indexing

During ingestion, standard temporal-heavy documents (e.g., stories, historical records) are processed into hierarchical chunks. These chunks are passed to two distinct pipelines:

- 1) **Vector Store Setup:** Text chunks are embedded using high-dimensional dense transformers (e.g., text-embedding-3-small) and stored in a standard Vector Database (e.g., Qdrant or ChromaDB).
- 2) **Graphiti Setup:** Simultaneously, the chunks are analyzed by an LLM to extract Entities, Relationships, and Temporal Boundaries (Valid_From, Valid_To). These are ingested into a Neo4j or local Graphiti instance.

B. Parallel Retrieval Engine

When a user issues a time-sensitive query (e.g., “Who was the CEO in 2021 compared to now?”), the system bypasses sequential routing. Using multithreading (or Python’s `asyncio`), the system spawns two distinct retrieval processes:

- **Process A (Semantic Search):** Conducts k-NN retrieval on the Vector Database to fetch raw paragraph structures and verbose context chunks based strictly on cosine similarity.
- **Process B (Graphiti Search):** Queries the temporal knowledge graph using Graphiti’s traversal API to extract valid nodes and edges related to the entities in the query, returning structured, time-stamped facts.

C. Aggregation, Filtering & Ranking

Because semantic chunks often contain outdated context, the Aggregator cross-references the retrieved vector chunks against the active edges in the Graphiti subgraph. If Process A returns a chunk stating “Alex is the regional manager” but Process B’s graph indicates the edge [Alex]-is-[Regional Manager] expired in 2022, the chunk is either heavily down-ranked or appended with a metadata tag: *WARNING: TEMPORALLY EXPIRED*. Valid, active context and relevant historical context are concatenated into a unified structured prompt.

D. Generation & Evaluation

The reconciled context is passed to the generation LLM (e.g., GPT-4o-mini). Performance will be assessed via two distinct methodologies:

- 1) **LongMemEval:** To test the model’s ability to recall and synthesize long-range factual updates and avoid regressions.
- 2) **TruLens:** To measure *Groundedness* (whether all generated statements are backed by the retrieved context) and *Temporal Context Precision* (the ratio of temporally-valid chunks fed to the LLM versus successfully filtered expired chunks). Latency metrics will also be profiled to compare the parallel implementation against sequential counterparts.